Assignment 2 Advanced Gen AI

- Data Robot practical
- CTGAN
- Data Querying examples in SQLLITEONLINE
- Practical on colab - Using pandas , ydata , AutoViz and Gemini for data analysis
- Visualization- IBM Cognos - report, exploration and storyline
- Biggan

Mini Project 1 - Data Science with Generative AI

Assignment Overview

This assignment challenges you to build a complete end-to-end data science pipeline using Python and Generative AI assistance. You will choose a real-world dataset and take it from raw CSV all the way through to a working predictive model — just like a professional data scientist would.

Gen AI Role: Throughout this assignment you are encouraged to use Gen AI tools (ChatGPT, Gemini, Claude) to write prompts that generate code, explain concepts, and suggest approaches. Document every prompt you use and what the AI generated output only.

Part 1 — Dataset Selection

Choose any real-world dataset that has at least 300 rows and 8 columns and contains a clear target variable you want to predict. The dataset must be a CSV file.

Recommended Dataset Sources

- Kaggle — kaggle.com/datasets
- UCI Machine Learning Repository — archive.ics.uci.edu
- Google Dataset Search — datasetsearch.research.google.com
- Hugging Face Datasets — huggingface.co/datasets

Suggested Datasets (pick one or choose your own)

Dataset Name	Target to Predict	Where to Download
Boston Housing	House price (regression)	Kaggle — vikrishnan/boston-house-prices
Titanic	Survival (classification)	Kaggle — titanic

Heart Disease UCI	Disease presence (0/1)	UCI Repository
Iris Flower	Species (multi-class)	UCI Repository
Student Performance	Final grade G3	UCI — student+performance
Bank Marketing	Term deposit subscribed	UCI Repository
Your own choice	Any clear target	Any reliable source

What to Submit for Part 1

1. Name of the dataset and source URL
2. Number of rows and columns
3. The target variable you have chosen to predict
4. Brief reason (2-3 sentences) why you chose this dataset
5. Run df.head() and df.describe() and include the output screenshot

Part 2 — ETL Pipeline

ETL stands for Extract, Transform, Load. You will build an automated pipeline that takes your raw CSV file, cleans and processes it, and saves the final clean version into a "gold" folder — ready for analysis and modelling.

What is ETL? Extract = load the raw data. Transform = clean, fix, and reshape the data. Load = save the clean data to a destination folder. This is exactly what data engineers do in real companies like Swiggy, Flipkart, and HDFC Bank every single day.

Example Gen AI Prompt

"I have a dataset called [your dataset name] with columns [list columns].

Write a Python ETL pipeline that:

1. Loads the raw CSV from a raw/ folder
2. Drops duplicate rows
3. Handles missing values appropriately
4. Converts data types where needed
5. Saves the clean data to a gold/ folder as clean_data.csv"

Required ETL Steps to Include

1

Extract — Load Raw Data

Read the CSV file using pandas. Print the shape and first 5 rows. Create a raw/ folder and save the original file there.

2

Inspect — Understand the Data

Check df.info(), df.describe(), df.isnull().sum() and df.duplicated().sum(). Print all findings clearly.

3

Transform — Clean the Data

Remove duplicate rows. Handle missing values (fill with mean/median for numbers, mode for categories). Fix wrong data types. Rename columns if needed.

4

Validate — Confirm Clean Data

After cleaning confirm: zero nulls remaining, correct data types, no duplicates, reasonable value ranges.

5

Load — Save to Gold Folder

Create a gold/ folder. Save the cleaned DataFrame as clean_data.csv. Print confirmation with final shape.

Expected Folder Structure After ETL

```
your_project/
  raw/
    original_dataset.csv      <- raw untouched file
  gold/
    clean_data.csv            <- cleaned ready to use
  notebook.ipynb              <- your Colab notebook
```

Part 3 — Database Schema

After cleaning the data you will design a proper relational database schema. A schema defines how data is organized into tables with relationships between them. This is the skill every data engineer needs.

What is a Database Schema? A schema is like a blueprint for your database. It defines which columns go in which table, what data type each column holds, and how tables are connected through foreign keys. Well-designed schemas prevent data duplication and make queries much faster.

Example Gen AI Prompt

"I have a dataset with these columns: [list all columns].

Group these columns into logical tables and design a

normalized relational database schema. Write:

1. The CREATE TABLE SQL statements with proper data types
2. Primary key and foreign key constraints
3. CHECK constraints for value ranges
4. An ERD diagram description"

What to Complete for Part 3

6. Group all columns into 3 to 6 logical tables (do not put everything in one table)
7. Write the CREATE TABLE SQL for each table with correct data types
8. Add PRIMARY KEY to each table
9. Add FOREIGN KEY relationships connecting tables
10. Add at least 3 CHECK constraints for value validation
11. Draw or describe the ERD (Entity Relationship Diagram) showing how tables connect
12. Write a Python script using SQLite to create the database and insert your clean data

Example Schema Structure

```
CREATE TABLE STUDENT (
   student_id  INTEGER PRIMARY KEY AUTOINCREMENT,
   school_id   INTEGER NOT NULL,
   sex         TEXT    NOT NULL,
   age         INTEGER NOT NULL CHECK (age BETWEEN 15 AND 22),
   FOREIGN KEY (school_id) REFERENCES SCHOOL(school_id)
```

);

Part 4 — Exploratory Data Analysis

EDA is the process of understanding your data before building any model. You will perform four types of analysis, each building on the previous one. Use Gen AI to help you write code and interpret the results.

Why EDA Matters: A data scientist spends 60-70% of their time understanding and cleaning data, not building models. Good EDA reveals patterns, outliers, and relationships that directly guide which model to build and which features to use.

4A — Descriptive Statistics

Generate summary statistics that describe the overall shape of your data.

Gen AI Prompt:

"Write Python code to generate descriptive statistics for my dataset. Show the mean, median, standard deviation, min and max for all numerical columns. Also show value counts for all categorical columns. Explain what each number means in plain English."

Required outputs:

- df.describe() output for all numerical columns
- df.value_counts() for all categorical columns
- Count and percentage of missing values per column
- A written paragraph (minimum 5 sentences) interpreting the key statistics in plain English

4B — Univariate Analysis

Analyse each column individually. How is each variable distributed on its own?

Gen AI Prompt:

"Write Python code to perform univariate analysis on my dataset. For each numerical column show a histogram and boxplot. For each categorical column show a bar chart. Identify any outliers and skewed distributions."

Required outputs:

- Histograms for all numerical columns (at least 4 charts)

- Boxplots to identify outliers (at least 3 charts)
- Bar charts for all categorical columns (at least 3 charts)
- Written observation for each chart: what does this distribution tell you?

4C — Bivariate and Multivariate Analysis

Analyse how columns relate to each other, especially how they relate to your target variable.

Gen AI Prompt:

"Write Python code for bivariate and multivariate analysis on my dataset. Show how each feature relates to the target variable [your target]. Create a correlation heatmap, scatter plots for the top features vs target, and box plots comparing the target across categorical groups. Identify the 3 strongest predictors."

Required outputs:

- Correlation heatmap of all numerical columns using seaborn
- At least 3 scatter plots: top features vs target variable
- At least 3 box plots: categorical columns vs target variable
- Identify the top 3 strongest predictors of your target
- A 6-chart combined subplot dashboard (3x2 grid using plt.subplots)
- Written paragraph explaining the most important relationships found

4D — Hypothesis Testing

Generate and test at least 4 hypotheses about your data. A hypothesis is a theory you test with evidence.

Gen AI Prompt:

"Given my dataset with columns [list columns] and target variable [target], generate 5 testable hypotheses about what factors affect the target. For each hypothesis write the Python code to test it and interpret the result."

Required format for each hypothesis:

HYPOTHESIS 1:

"Students with higher study time will score better in G3"

Test code:

df.groupby("studytime")["G3"].mean()

Result: studytime=1 avg G3=9.5, studytime=4 avg G3=12.4

Conclusion: CONFIRMED - more study time leads to higher grades

Part 5 — Predictive Model Development

Now build and evaluate a machine learning model to predict your target variable. Follow all 5 stages below. Each stage uses Gen AI to help you write better code and understand the results.

5A — Model Architecture Selection

Compare at least 3 different model types and justify which one you select for your final model.

Gen AI Prompt:

"I have a dataset predicting [target variable]. My target is [continuous/categorical]. Compare Linear Regression, Random Forest, and Gradient Boosting for this task. Write Python code to train all three, evaluate them using cross-validation R2 score or accuracy, and recommend which one to use and why."

Required outputs:

- Code to train Linear Regression (or Logistic Regression for classification)
- Code to train Random Forest
- Code to train Gradient Boosting
- Comparison table showing R2 / accuracy for all three models
- Written justification: which model did you select and exactly why?

5B — Feature Importance Assessment

Identify which columns (features) are most important for making predictions.

Gen AI Prompt:

"Using my trained Random Forest model on the [dataset name] dataset, write Python code to calculate feature importance scores. Plot a horizontal bar chart of the top 10 features sorted by importance. Explain what the top 3 most important features tell us about the data."

Required outputs:

- Feature importance scores for all columns
- Horizontal bar chart of top 10 features
- Identify which features can be removed (importance below 0.01)
- Written explanation of the top 3 most important features and why they matter

5C — Ensemble Model Creation

Combine multiple models into a stacking ensemble to get better prediction accuracy.

Gen AI Prompt:

"Create a stacking ensemble model combining Linear Regression (or Logistic Regression), Random Forest, and Gradient Boosting for my [dataset name] prediction task. Use Ridge (or Logistic Regression) as the meta-model. Compare the ensemble R2 score and MAE against each individual model. Show the results in a formatted table."

Required outputs:

- StackingRegressor or StackingClassifier combining 3 base models
- Ridge or Logistic Regression as the final meta-model
- Performance comparison table: individual models vs ensemble
- Written explanation: did the ensemble improve over the best single model? Why or why not?

5D — Model Interpretability

Explain why the model made specific predictions using SHAP values.

Gen AI Prompt:

"Install SHAP and write Python code to explain my Random Forest model predictions using SHAP values. Show a SHAP summary plot for all features. Then explain the prediction for one specific row in plain English — list which features pushed the prediction up and which pushed it down."

Required outputs:

- SHAP summary plot (beeswarm plot) for all features
- SHAP force plot for at least one individual prediction
- Written explanation in plain English: why did the model predict that value?
- Which features had the largest positive and negative SHAP impact?

5E — Improved Generalisation

Check if your model is overfitting and use cross-validation and hyperparameter tuning to improve it.

Gen AI Prompt:

"Write Python code to check if my Random Forest model is overfitting. Show train R2 vs test R2. Then run 5-fold cross-validation and report the mean and standard deviation. Then use GridSearchCV to find the best hyperparameters. Show a before vs after comparison table of train score, test score, and overfit gap."

Required outputs:

- Train R2 vs Test R2 — is there an overfitting gap?
- 5-fold cross-validation scores (mean and standard deviation)
- GridSearchCV with at least 3 hyperparameters and their possible values
- Before vs After tuning comparison table
- Final conclusion: which model configuration generalises best?

Part 6 — Create Prediction from the Final model

Mini Project 2 - Software Development Using Generative AI

Assignment Overview

You are free to choose any real-world application for this assignment. Examples include: a hospital appointment system, a college library management app, a food delivery tracker, an online exam portal, a hostel room allocation system — or any application of your own choice.

For your chosen application, you will complete all 7 phases below using Generative AI at every step. Each phase uses a specific tool. Your final submission must include screenshots of your prompts and outputs from every phase, added to assignment 2 document.

Tool	Phase it is Used In
ChatGPT / AI Classroom	Phase 1 — Requirements Analysis
Visily (visily.ai)	Phase 2 — Wireframe Design
Eraser.io	Phase 3 — Architecture and Database Design
Google Colab	Phases 4, 5, 6, 7 — Development through Maintenance

PHASE 1

Requirements Analysis

Tool: ChatGPT / AI Classroom

Use Generative AI to analyse the requirements for your chosen application. Your submission for this phase must include:

- Raw user feedback — write 10 to 15 realistic feedback messages from imaginary users of your application
- Requirements table — use Gen AI to extract and structure requirements with ID, description, category, and priority
- User stories — convert your high-priority requirements into user story format
- Non-functional requirements — identify at least 5 non-functional requirements relevant to your application

Screenshot all prompts you used and all outputs from Gen AI.

PHASE 2

Wireframe Design

Tool: Visily — app.visily.ai

Use Visily's AI wireframe feature to design the screens of your application. Your submission for this phase must include:

- Home screen wireframe — the main landing page of your application
- Main action screen wireframe — the primary user action (e.g. booking form, order page, registration form)
- Dashboard / summary screen wireframe — showing the user's current status and history
- A short explanation of how the three screens connect and what each screen does

Screenshot your prompts in Visily and the generated wireframes.

PHASE 3

Architecture and Database Design

Tool: Eraser.io

Use Eraser.io's AI diagram feature to design the technical structure of your application. Your submission for this phase must include:

- System architecture diagram — showing Frontend, Backend, Database, and any External Services
- Entity Relationship (ER) diagram — showing all database tables, their fields, primary keys, foreign keys, and relationships
- Database CREATE TABLE statements — generated using Gen AI based on your ER diagram
- A short explanation of your database tables and why each one exists

Screenshot your prompts and the diagrams generated by Eraser.io.

PHASE 4

Development

Tool: Google Colab

Use Gen AI to generate Python code and build your application in Google Colab. Complete all four modules below in separate Colab cells, run each one, and screenshot the output.

Module 1 — Database Setup

Use Gen AI to create your database, create all tables, and insert sample data. Verify by printing the record count from each table.

Module 2 — Core Data Functions

Use Gen AI to write search, filter, and list functions for your main data. Test each function and print the results.

Module 3 — Main Business Function

Use Gen AI to write the central function of your application — for example: place a booking, submit an order, or register a user. The function must validate inputs, write to the database, update related records, and return a confirmation.

Module 4 — HTML Frontend

Use Gen AI to generate a simple HTML page displayed inside Colab using IPython.display.HTML. The page must load data from your database and show a product/item grid, a user status section, and a styled header and footer.

Your submission must include: screenshots of each module's output, all prompts used, and your complete Python code.

PHASE 5

Testing

Tool: Google Colab

Use Gen AI to generate and run an automated test suite for your application in Google Colab. Your test suite must cover four groups:

- Database structure tests — verify all tables exist
- Data quality tests — check for invalid values such as negative prices, null names, or out-of-range fields
- Business logic tests — verify your main function worked correctly and updated related records
- Edge case tests — test unusual inputs such as searching for something that does not exist or entering duplicate data

The output must show a formatted pass or fail result for each test and a total pass rate. Screenshot the complete test report.

PHASE 6

Deployment and Monitoring

Tool: Google Colab

Use Gen AI to simulate deployment monitoring and a CI/CD pipeline for your application in Google Colab. Your submission must include:

- Traffic monitoring simulation — 10 minutes of API response time data with a spike injected at minutes 4 to 6, threshold alerts, and AI automated actions

Screenshot outputs and include all prompts used.

PHASE 7

Maintenance and Prediction

Tool: Google Colab

Use Gen AI to simulate 30 days of server metrics and predict future issues for your application in Google Colab. Your submission must include:

- A table showing the last 7 days of metrics: daily users, error rate, database load, and response time
- Predictions of how many days until each metric hits a critical threshold
- A prioritised list of at least 5 maintenance recommendations with urgency levels
- A short reflection paragraph: which prediction concerns you most and why

Screenshot all outputs and include the prompt you used.

Submission Instructions

Compile all your work into a single PDF file and submit it. Your PDF must be organised in phase order. For every phase include:

1. The prompt or prompts you typed into the Gen AI tool
2. A screenshot of the output from the Gen AI tool
3. Any code generated — paste it or screenshot it
4. A screenshot proving the code ran successfully in Colab

Important

- Choose any application — your choice is completely open
- All prompts must be customised for your chosen application
- All code must run successfully — screenshots are required as proof
- Work must be original — do not copy another student's application or prompts
- Submit as a single PDF compiled in phase order